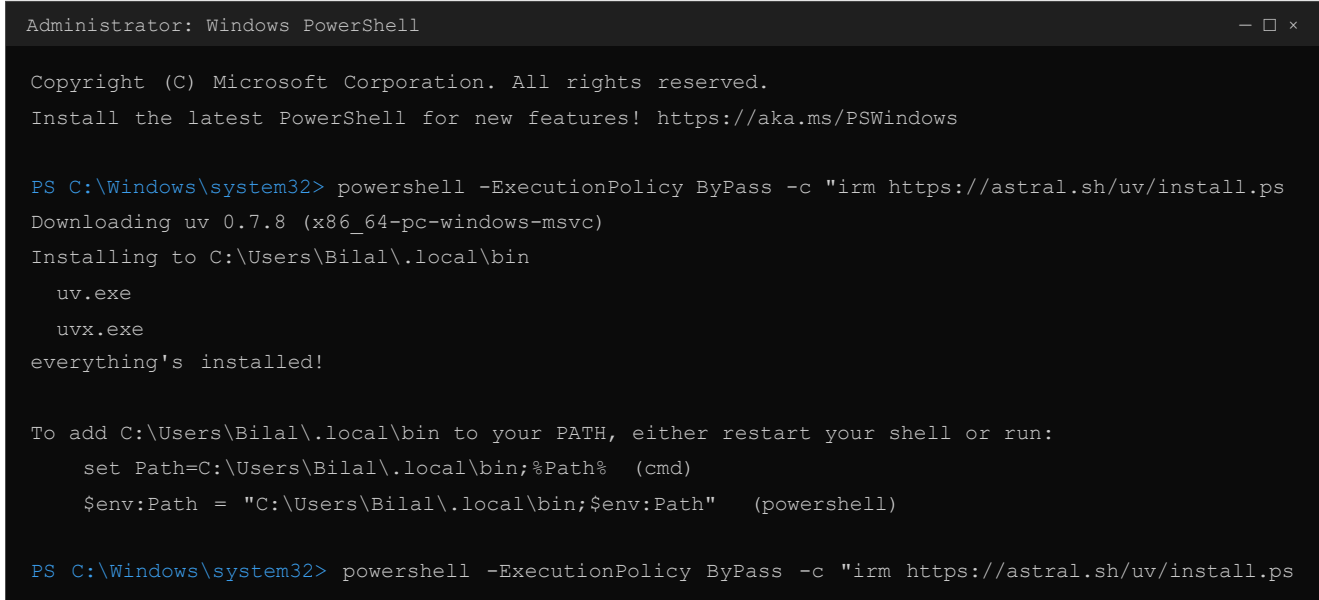


Artificial Intelligence Practical Work:

Problem 1: How to install OpenAI SDK and create a first AI agent:

Step 1: Install UV for this command in PowerShell:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/
install.ps1 | iex"
```



```
Administrator: Windows PowerShell

Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features! https://aka.ms/PSWindows

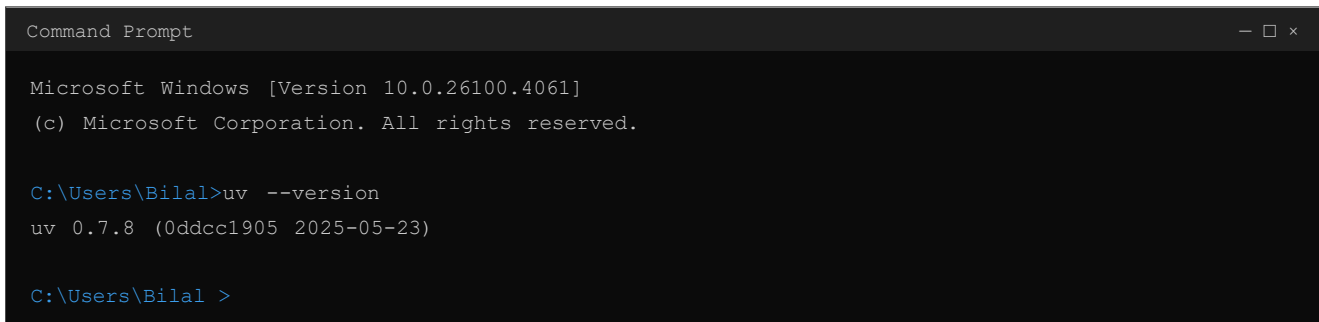
PS C:\Windows\system32> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps
Downloading uv 0.7.8 (x86_64-pc-windows-msvc)
Installing to C:\Users\Bilal\.local\bin
  uv.exe
  uvx.exe
everything's installed!

To add C:\Users\Bilal\.local\bin to your PATH, either restart your shell or run:
  set Path=C:\Users\Bilal\.local\bin;%Path%    (cmd)
  $env:Path = "C:\Users\Bilal\.local\bin;$env:Path"    (powershell)

PS C:\Windows\system32> powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps
```

Step 2: Check UV version for this command:

```
uv -version
```



```
Command Prompt

Microsoft Windows [Version 10.0.26100.4061]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Bilal>uv --version
uv 0.7.8 (0ddcc1905 2025-05-23)

C:\Users\Bilal >
```

Step 3: Create Folder

Step 4: Open folder in VS code

Step 5: Initialized the project for this command:

```
uv init .
```

OR

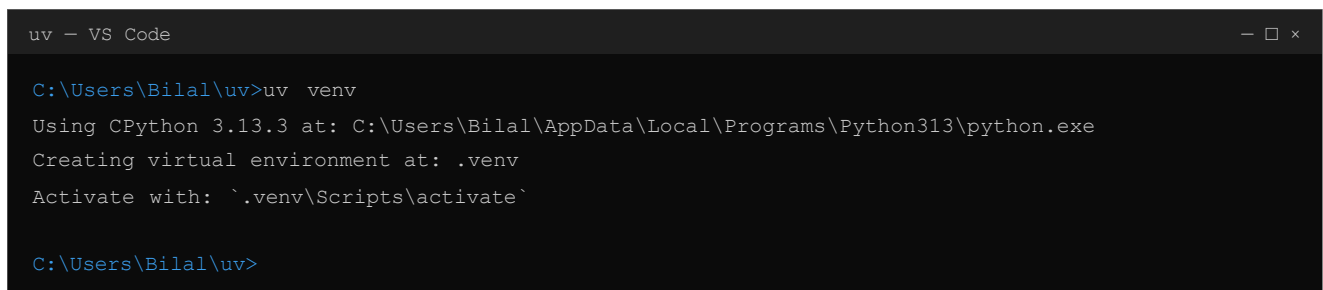
```
uv init [FOLDER NAME]
```

A terminal window titled 'uv - VS Code' with a dark background. The prompt is 'C:\Users\Bilal\uv>'. The command 'uv init .' is entered, followed by the output 'Initialized project `uv` at `C:\Users\Bilal\uv`'. The prompt returns to 'C:\Users\Bilal\uv>'.

```
uv - VS Code
C:\Users\Bilal\uv>uv init .
Initialized project `uv` at `C:\Users\Bilal\uv`
C:\Users\Bilal\uv>
```

Step 6: Create Virtual Environment for this command:

```
uv venv
```

A terminal window titled 'uv - VS Code' with a dark background. The prompt is 'C:\Users\Bilal\uv>'. The command 'uv venv' is entered, followed by the output 'Using CPython 3.13.3 at: C:\Users\Bilal\AppData\Local\Programs\Python313\python.exe', 'Creating virtual environment at: .venv', and 'Activate with: `.venv\Scripts\activate`'. The prompt returns to 'C:\Users\Bilal\uv>'.

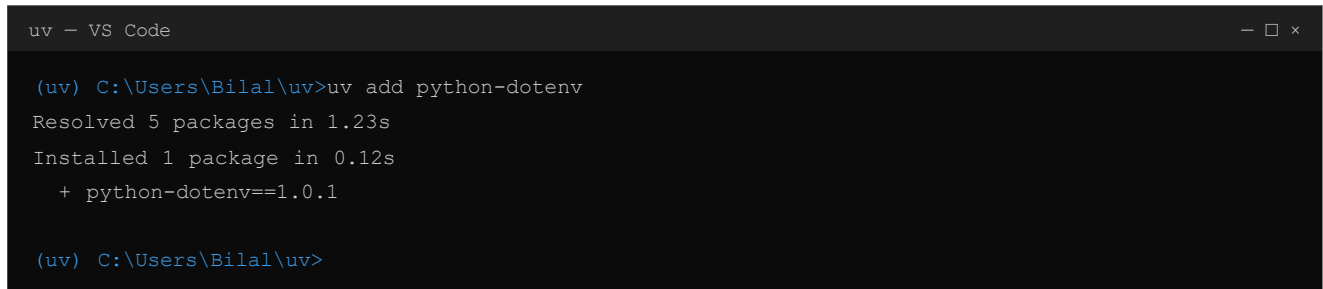
```
uv - VS Code
C:\Users\Bilal\uv>uv venv
Using CPython 3.13.3 at: C:\Users\Bilal\AppData\Local\Programs\Python313\python.exe
Creating virtual environment at: .venv
Activate with: `.venv\Scripts\activate`
C:\Users\Bilal\uv>
```

Step 7: Activate Virtual Environment:

```
.venv\Scripts\activate
```

Step 8: Install Python Env Package in cmd:

```
uv add python-dotenv
```

A screenshot of a terminal window titled "uv - VS Code". The terminal shows the command "(uv) C:\Users\Bilal\uv>uv add python-dotenv" being executed. The output shows "Resolved 5 packages in 1.23s" and "Installed 1 package in 0.12s", followed by "+ python-dotenv==1.0.1". The prompt returns to "(uv) C:\Users\Bilal\uv>".

```
uv - VS Code

(uv) C:\Users\Bilal\uv>uv add python-dotenv
Resolved 5 packages in 1.23s
Installed 1 package in 0.12s
+ python-dotenv==1.0.1

(uv) C:\Users\Bilal\uv>
```

Step 9: Create a deepseek API Key:

The image shows two screenshots of the DeepSeek Developer Platform website. The top screenshot is the main landing page for developers, with the URL 'platform.deepseek.com' in the browser's address bar. It features the 'DeepSeek Developer Platform' header, a 'Get a DeepSeek API Key' button, and the title 'DeepSeek Developer API'. Below the title, it says 'Get a DeepSeek API key and make your first API request in minutes.' The bottom screenshot shows the 'API Keys' section of the platform, with the URL 'platform.deepseek.com/api_keys'. It displays a generated API key 'sk-NDTbLbpsXmTEsUCioM5DhulN8ItTghLk' with a 'Copy' button. A warning message states: 'Use your API keys securely. Do not share them or embed them in code the public can view.' At the bottom of this section is a '+ Create API key' button.

platform.deepseek.com

DeepSeek Developer Platform

Get a DeepSeek API Key

DeepSeek Developer API

Get a DeepSeek API key and make your first API request in minutes.

platform.deepseek.com/api_keys

DeepSeek Platform

API Keys

API key generated

sk-NDTbLbpsXmTEsUCioM5DhulN8ItTghLk [Copy](#)

Use your API keys securely. Do not share them or embed them in code the public can view.

+ Create API key

Step 10: Create .env file in root folder and Apply API Key in .env file:

```
DEEPSEEK_API_KEY = '${API_KEY}'
```

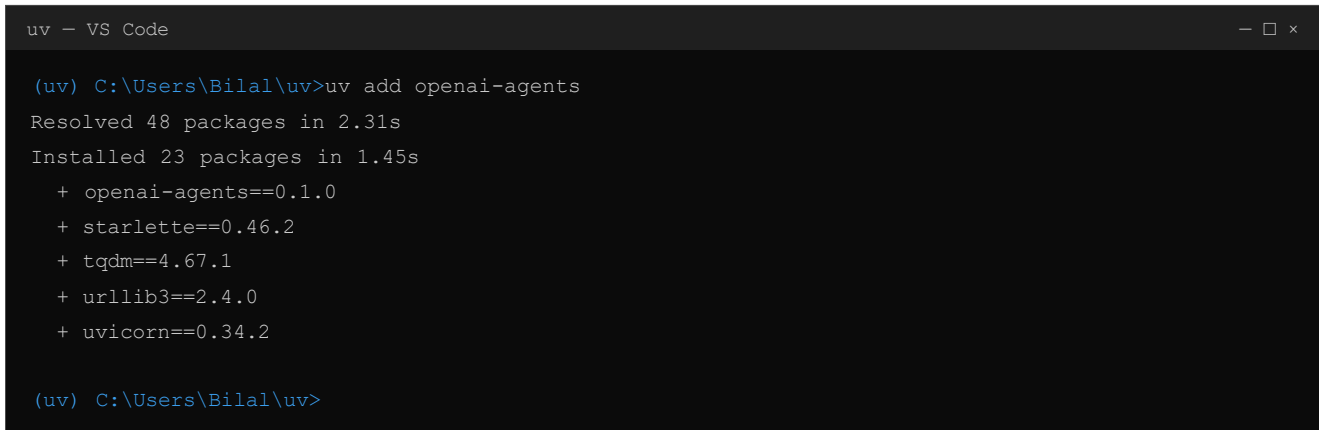
A screenshot of a VS Code editor window. The title bar at the top says "uv - VS Code". The editor area contains a single line of text: `DEEPSEEK_API_KEY='${API_KEY}'`. The text is in a light green color on a dark background.

```
uv - VS Code

DEEPSEEK_API_KEY='${API_KEY}'
```

Step 11: Install OpenAI Agent:

```
uv add openai-agents
```

A screenshot of a terminal window. The title bar at the top says "uv - VS Code". The terminal shows the command `(uv) C:\Users\Bilal\uv>uv add openai-agents` being executed. The output shows that 48 packages were resolved in 2.31s and 23 packages were installed in 1.45s. A list of installed packages follows: `+ openai-agents==0.1.0`, `+ starlette==0.46.2`, `+ tqdm==4.67.1`, `+ urllib3==2.4.0`, and `+ uvicorn==0.34.2`. The prompt `(uv) C:\Users\Bilal\uv>` is shown at the bottom.

```
uv - VS Code

(uv) C:\Users\Bilal\uv>uv add openai-agents
Resolved 48 packages in 2.31s
Installed 23 packages in 1.45s
+ openai-agents==0.1.0
+ starlette==0.46.2
+ tqdm==4.67.1
+ urllib3==2.4.0
+ uvicorn==0.34.2

(uv) C:\Users\Bilal\uv>
```

Step 12: Create Your First Agent:

Insert the code in main.py file

```
from agents import Agent, AsyncOpenAI, OpenAIChatCompletionsModel, RunConfig, Runner
from dotenv import load_dotenv
import os

load_dotenv(override=True)
deepseek_api_key = os.getenv("DEEPSEEK_API_KEY")

#check if the API key is present; if not, raise an error
if not deepseek_api_key:

    raise ValueError("DEEPSEEK_API_KEY is not set. Please ensure it is defined in

#Reference: https://platform.deepseek.com
external_client = AsyncOpenAI(

    api_key=deepseek_api_key,
    base_url="https://api.deepseek.com/v1"
)

model = OpenAIChatCompletionsModel(

    model = "deepseek-chat",
    openai_client = external_client
)

config = RunConfig(

    model=model,
    model_provider = external_client,
    tracing_disabled = True
)

agent = Agent(

    name = "Translator",
    instructions="You are a helpful translator. Always translate English Sentences
)

response = Runner.run_sync(

    agent,
    input= 'input= "My name is Bilal."',
    run_config= config
)

print(response)
```

Step 13: Execute/Run the Agent

```
uv run main.py
```



```
C:\Windows\System32\cmd.exe

D:\SMIU work\Spring 2026\AI\Lectures\New folder>uv run main.py
- Last agent: Agent(name="Translator", ...)
- Final output (str):
    (Mera naam Bilal hai.) !بلال خوشی ہے کہ آپ سے ملاقات ہوئی، حسن
- 1 new item(s)
- 1 raw response(s)
- 0 input guardrail result(s)
- 0 output guardrail result(s)
(See 'RunResult' for more details)

D:\SMIU work\Spring 2026\AI\Lectures\New folder>
```

Note: You can try using different sentences and paragraphs.

You can also create your own agent for a specific purpose. The agent can process your queries and provide responses quickly.

Problem 2: Build an AI Chatbot Using Chainlit:

Step 1: Create new folder and same process to initialize the project

Step 2: Open the chainlit document:

<https://docs.chainlit.io/get-started/overview>

You can install Chainlit via pip, but I used UV

```
uv add chainlit
```



```
simple_chatbot - VS Code

(simple_chatbot) C:\Users\Bilal\simple_chatbot>uv add chainlit
Resolved 52 packages in 3.21s
Installed 29 packages in 2.14s
+ chainlit==2.5.5
+ watchfiles==0.20.0
+ wrapt==1.17.2
+ wsproto==1.2.0
+ yamll==1.20.1
+ zipp==3.23.0

(simple_chatbot) C:\Users\Bilal\simple_chatbot>
```


Step 3: You can check.

```
simple_chatbot - VS Code

pyproject.toml
[project]
name = "simple-chatbot"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "chainlit>=2.5.5",
]
```

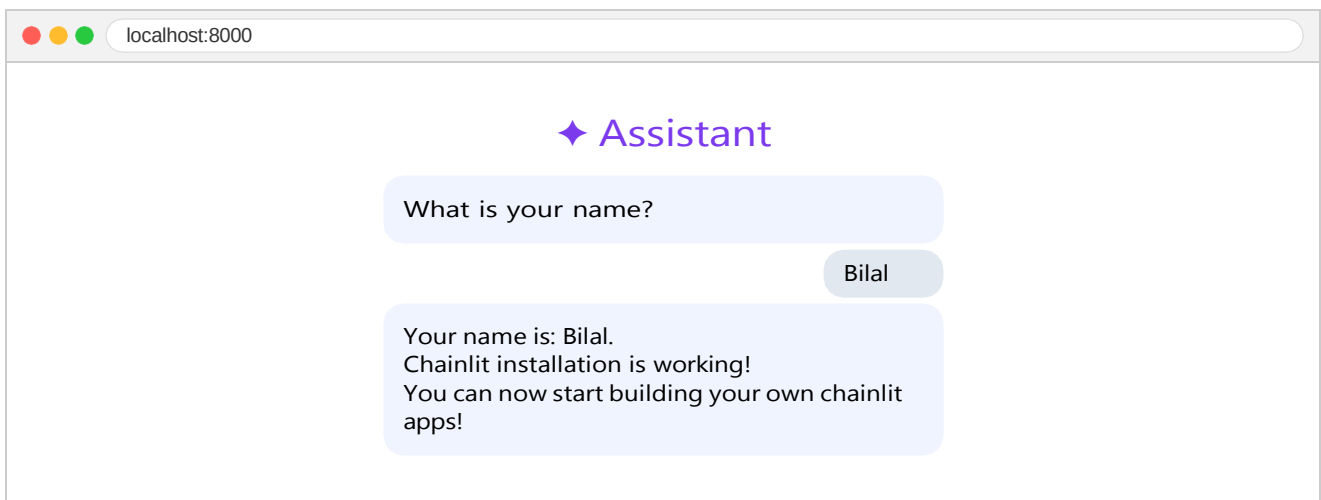
Step 4: Make sure everything run smoothly:

Chainlit hello

Output: <http://localhost:8000>

```
simple_chatbot - VS Code

(chainlit) C:\Users\Bilal\simple_chatbot>chainlit hello
```



Step 5: Few things are most important you write the code:

Decorators:

<code>@on_chat_start</code>	means	New chat session
<code>@on_message</code>	means	New message from the user
<code>@on_stop</code>	means	user clicks the stop button
<code>@on_chat_resume</code>	means	user continues a chat session
<code>@on_chat_end</code>	means	user disconnected or started new session

Step 6: Write your own code in main.py file

```
import chainlit as cl

@cl.on_message
async def main(message: cl.Message):

    response = f"Received: {message.content}"
    await cl.Message(content = response).send()
```

Step 7: Run your project:

```
Chainlit run main.py -w
```



The **-w** flag enables auto-reloading, meaning the application will automatically restart whenever changes are made to the files.

There are two types of conversations: **stateless** and **stateful**.

In a **stateless** conversation, the system does not store any history, so it cannot remember previous interactions (for example, it cannot recall your name).

Now we will build a stateful conversation project, where the chatbot can remember previous messages and provide more accurate responses.

Problem 3: Build your Own ChatGPT with Python

Step 1: Create new folder and same process to initialize the project

Step 2: Install Libraries:

```
Uv add openai-agents python-dotenv
```

Step 3: Install Chainlit

```
Uv add chainlit
```

Step 4: See all dependencies in pyproject.toml file

```
my_chatgpt - VS Code

pyproject.toml
[project]
name = "my-chatgpt"
version = "0.1.0"
requires-python = ">=3.12"
dependencies = [
    "chainlit>=2.5.5",
    "openai-agents>=0.1.0",
    "python-dotenv>=1.0.1",
]
```

Step 5: Create .env file in root folder and paste the DeepSeek api key

```
DEEPSEEK_API_KEY = '${API_KEY}'
```

Step 6: Write the code in main.py

```
import os
from dotenv import load_dotenv
from typing import cast
import chainlit as cl
from agents import Agent, Runner, AsyncOpenAI, OpenAIChatCompletionsModel, OpenAIP
from agents.run import RunConfig
```

```

load_dotenv(override=True)
deepseek_api_key = os.getenv("DEEPSEEK_API_KEY")

#check if the API key is present; if not, raise an error
if not deepseek_api_key:

    raise ValueError("DEEPSEEK_API_KEY is not set. Please ensure it is defined in

@cl.on_chat_start
async def start():

    #Reference: https://platform.deepseek.com
    external_client = AsyncOpenAI(
        api_key=deepseek_api_key,
        base_url="https://api.deepseek.com/v1"
    )
    model_instance = OpenAIChatCompletionsModel(
        model = "deepseek-chat",
        openai_client = external_client
    )
    provider_instance = OpenAIProvider(
        openai_client=external_client
    )
    config = RunConfig(
        model=model_instance,
        model_provider=provider_instance,
        tracing_disabled = True
    )
    cl.user_session.set("chat_history", [])
    cl.user_session.set("config", config)

    agent: Agent = Agent(
        name = "Assistant",
        instructions = "You are a helpful assistant.",
        model = model_instance
    )
    cl.user_session.set("agent", agent)

    await cl.Message(content = "Welcome to the SMIU AI Assistant! How can I help y

@cl.on_message
async def main(message: cl.Message):

    msg = cl.Message(content = "Thinking...")

```

```

await msg.send()

agent: Agent = cast(Agent, cl.user_session.get("agent"))
config: RunConfig = cast(RunConfig, cl.user_session.get("config"))
history = cl.user_session.get("chat_history") or []
history.append({"role": "user", "content": message.content})

try:
    print("
[CALLING_AGENT_WITH_CONTEXT]
", history, "
")

    result = await Runner.run(
        starting_agent = agent,
        input = history,
        run_config = config
    )

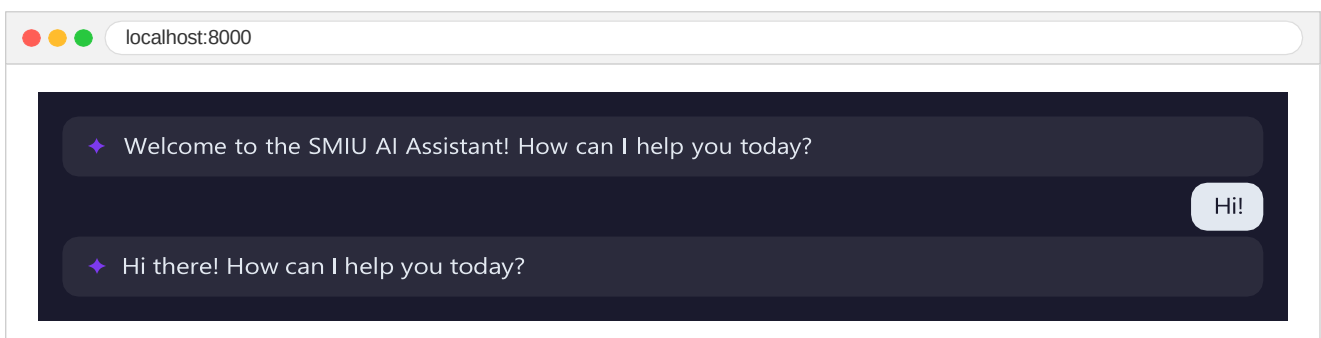
    response_content = result.final_output
    msg.content = response_content
    await msg.update()

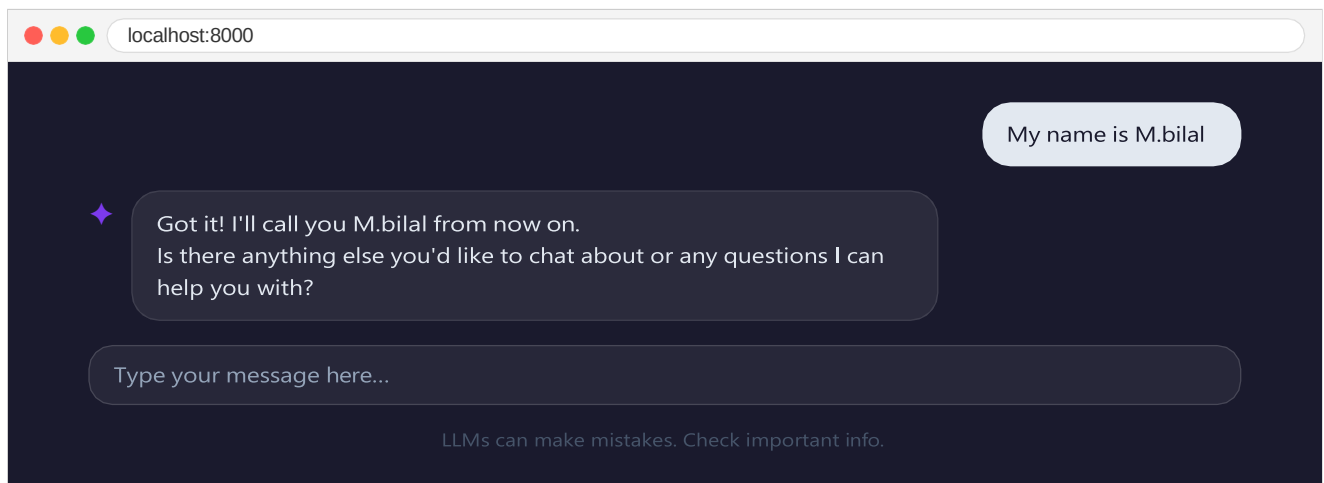
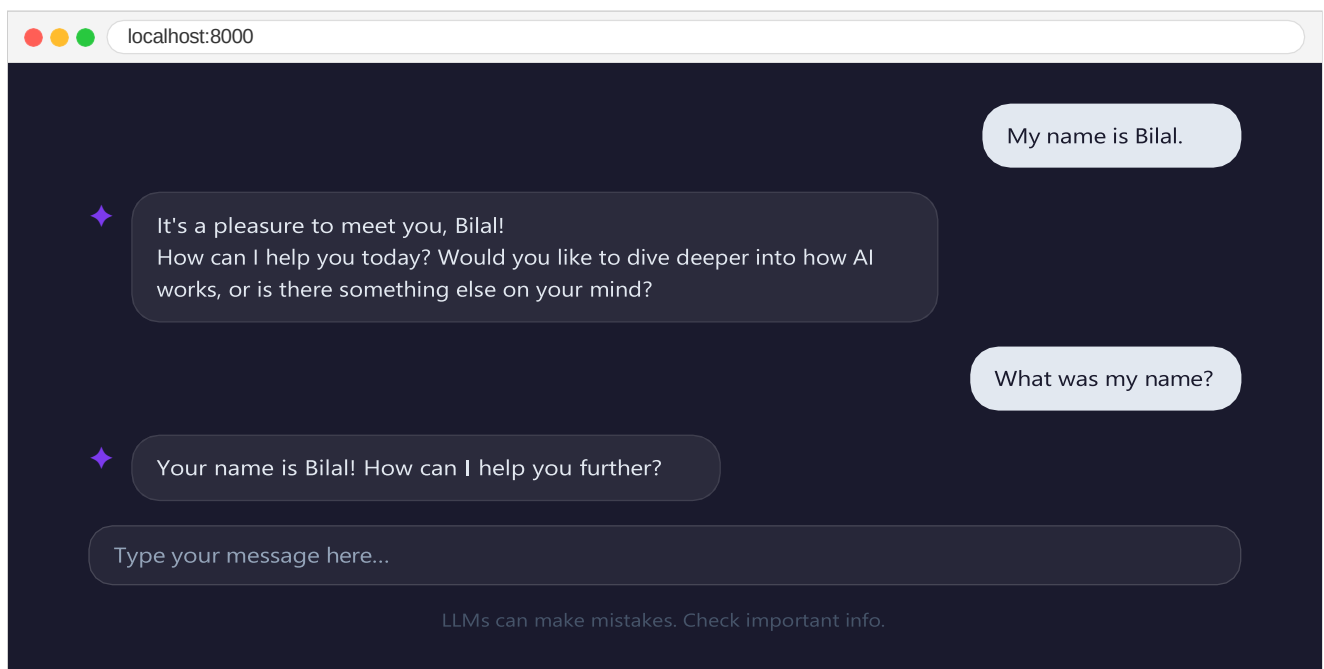
    cl.user_session.set("chat_history", result.to_input_list())
    print(f"User: {message.content}")
    print(f"Assistant: {response_content}")
except Exception as e:
    msg.content = f"Error: {str(e)}"
    await msg.update()
    print(f"Error: {str(e)}")

```

Step 7: Run the project:

```
chainlit run main.py
```





As you can see, the system is storing previous conversation history and can recall it when asked. This is called a **stateful conversation**.

This AI agent can answer your questions more effectively by using past interactions.

Problem 4: Tool Calling in Agentic AI SDK

What is tool calling?

Tool calling in OpenAI is a capability allowing models (like GPT-4) to connect with external tools, APIs, or databases to fetch real-time data or perform actions.

Types of Tool Calling:

1. **Function Calling (Custom Tools):** You define custom Python functions or API endpoints using a JSON schema, which the model can call to execute code, retrieve data, or act on your behalf.
2. **Built-in Tools (Hosted Tools):** OpenAI-managed tools that run alongside the model on their servers, including:
 - **Web Search:** Enables the model to browse the internet for up-to-date information.
 - **File Search (RAG):** Allows the model to retrieve information from uploaded documents (Vector Stores).
 - **Code Interpreter:** Lets the model run Python code in a sandboxed environment for calculations or data analysis.
 - **Image Generation:** Generates images from user prompts.
3. **Local Runtime Tools:** Tools that run in your own application environment but are triggered by the model, such as the ComputerTool for GUI/browser automation.

LLMs (Large Language Models) only have access to historical data. They do not have real-time or personalized data by default. To access such data, we use function calling or tool calling mechanisms.

Step 1: Create new folder and same process to initialize the project

Step 2: Install Libraries:

```
uv add openai-agents python-dotenv
```

Step 3: Create .env file in root folder and paste the DeepSeek api key

```
DEEPSEEK_API_KEY = '${API_KEY}'
```

Step 4: Write the code in main.py

```
from agents import Agent, AsyncOpenAI, OpenAIChatCompletionsModel, RunConfig, Runn
from dotenv import load_dotenv
import os

load_dotenv(override=True)
deepseek_api_key = os.getenv("DEEPSEEK_API_KEY")

#check if the API key is present; if not, raise an error
if not deepseek_api_key:

    raise ValueError("DEEPSEEK_API_KEY is not set. Please ensure it is defined in
```

```

#Reference: https://platform.deepseek.com
external_client = AsyncOpenAI(

    api_key=deepseek_api_key,
    base_url="https://api.deepseek.com/v1"
)
model = OpenAIChatCompletionsModel(

    model = "deepseek-chat",
    openai_client = external_client
)
config = RunConfig(

    model=model,
    model_provider = external_client,
    tracing_disabled = True
)

```

Step 5: Create a file tool.py

```

from agents import Agent, Runner
from main import config

agent = Agent(

    name = 'General Agent',
    instructions = "You are a helpful assistant. Your task is to help to user with
)

result = Runner.run_sync(

    agent,
    "Who is the founder of Pakistan?",
    run_config = config
)

print(result.final_output)

```

Step 5: Run the project:

```
uv run tool.py
```

C:\Windows\System32\cmd.exe

— □ ×

```
D:\SMIU work\Spring 2026\AI\Lectures\New folder>uv run tool.py
```

Quaid-e-Azam Muhammad Ali Jinnah is the founder of Pakistan.

He was a lawyer, politician, and statesman who is revered as the "Father of the Nation" in Pakista

He led the All-India Muslim League and played a central role in the creation of Pakistan, which came into existence on August 14, 1947.

```
D:\SMIU work\Spring 2026\AI\Lectures\New folder>uv run tool.py
```

The current date is approximately late 2024 to early 2025.

However, I don't have access to real-time information, so I cannot provide the exact current date.

```
D:\SMIU work\Spring 2026\AI\Lectures\New folder>
```

You can observe that the model provides correct answers for historical information, but it is not trained on real-time data. Therefore, it may generate approximate or assumed answers for current information (such as the current date). Typically, LLMs are trained on data available up to a certain cutoff point.

Now, let's understand what Context Management is.

Context Management: Context management in the OpenAI Agent SDK refers to handling additional data that your code can use during an agent's execution. This "context" comes in two main forms:

1. **Agent/LLM context:** Refers to the information the LLM sees when it generates responses. This is essentially the conversation history or messages (like system prompts, instructions, and user inputs) that guide its output.
2. **Local Context:** This is represented via the `RunContextWrapper` class and the `context` property within it. The way this works is:
 1. You create any Python object you want. A common pattern is to use a dataclass or a Pydantic object.
 2. You pass that object to the various run methods (e.g. `Runner.run(..., context=whatever)`).
 3. All your tool calls, lifecycle hooks etc will be passed a wrapper object, `RunContextWrapper[T]`, where `T` represents your context object type which you can access via `wrapper.context`.

Let's start the Local Context work:

Step 7: Write the code in tool.py

```
from agents import Agent, Runner, function_tool, RunContextWrapper
import asyncio
from connection import config
from dataclasses import dataclass

@dataclass
class UserInfo:

    name: str
    uid: int

@function_tool
async def fetch_user_age(wrapper: RunContextWrapper[UserInfo]) -> str:
```

```

        return f"User {wrapper.context.name} is 25 years old."
# this is Local context

async def main():

    user_info = UserInfo(name = "Bilal", uid = 66)

    agent = Agent[UserInfo](

        name = "Assistant",
        # instructions = "use the `fetch_user_age` tool to get the user's age.",
        tools = [fetch_user_age]
    )

    result = await Runner.run(

        starting_agent = agent,
        input = "What is the age of the user?",
        context = user_info,
        run_config = config
    )

    print(result.final_output)

if __name__ == "__main__":

    asyncio.run(main())

```

You can run the code both with and without instructions. This will help you clearly understand the concept of tool calling.

Step 8: Run the project.

Uv run tool.py

In the same way, you can create multiple tools to automate and handle your personal tasks.

Problem 5: Handoffs Agent

What is Handoffs?

In simple terms, it means transferring a task from one agent to another. Just like people specialize in different fields, if a task is beyond the capability of the main (general) agent, it will hand off the task to a specialized agent.

Step 1: Create new folder and same process to initialize the project

Step 2: Create a file connection.py and write the connection code.

Step 3: Write the code in main.py

```
from agents import Agent, Runner, handoff
import asyncio
from connection import config

billing_agent = Agent(

    name = "Billing Agent",
    instructions = "You only answer questions related to billing."
)

refund_agent = Agent(

    name = "Refund Agent",
    instructions = "You are responsible only for handling refund-related requests"
)

custom_refund_handoff = handoff(

    agent = refund_agent,
    tool_name_override = "custom_refund_tool",
    tool_description_override = "Handle user refund requests with extra care."
)

damage_refund_handoff = handoff(

    agent = refund_agent,
    tool_name_override = "damage_refund_tool",
    tool_description_override = "Handle refund due to damaged item."
)

late_delivery_refund_handoff = handoff(

    agent = refund_agent,
    tool_name_override = "late_delivery_refund_tool",
    tool_description_override = "Handle refund due to late delivery."
)
```

```

triage_agent = Agent(

    name = "Triage Agent",
    instructions = "Read the user's request and decide which agent is best suited

    handoffs = [billing_agent, custom_refund_handoff, damage_refund_handoff,
    late_delivery_refund_handoff]
)

async def main():

    result = await Runner.run(

        triage_agent,

        # "I need a refund for my recent purchase.",

        "My order arrived 10 days late. I want a refund."
        run_config = config
    )
    print(result.final_output)

if __name__ == "__main__":

    asyncio.run(main())

```

Step 4: Run the project

Uv run handoffs.py